

AQUARIUM GENETICS GAME

Unity Implementation Design Plan

A complete guide for LLM-assisted development

How To Use This Document

This plan is structured so you can paste each section directly into an LLM coding assistant (like Claude, GPT-4, or Cursor AI) and receive working Unity C# code. Each section is self-contained.

Icons used throughout:

- ☐ YOU DO THIS — tasks that genuinely require human action (external accounts, art creation, certificates)
- ☒ AUTOMATED — generate an EditorScript with your LLM, then run it from the Tools menu; you still click Run but the LLM writes all the code
- ⚠ IMPORTANT — critical notes that prevent common mistakes
- Code blocks — paste these directly as LLM prompts, or implement as shown

⚠ Always work in Unity 6 (6000.x). Create a new Universal 2D project before starting. When prompting your LLM, include: "Use Unity 6 (6000.x) compatible APIs."

☐ **YOU DO THIS:** Create a new Unity project: File > New Project > Universal 2D (not "2D Built-In"). Name it 'AquariumGenetics'. Use Unity 6 (6000.x).

Phase 1: Project Structure & Folder Setup

Ask your LLM:

```
"Create the following folder structure under Assets/ in a Unity project:  
Scripts/Genetics, Scripts/Fish, Scripts/Rendering, Scripts/GameLoop,  
Scripts/UI, Scripts/Services, ScriptableObjects/Genes,  
Shaders/, Prefabs/Fish, Prefabs/UI, Scenes/. Just the folder creation  
script."
```

Then ask:

```
"Create a Unity EditorScript at Editor/ProjectSetup.cs that creates all those  
folders via AssetDatabase.CreateFolder when run from a menu item Tools >  
Setup Project."
```

☐ **YOU DO THIS:** Run the menu item Tools > Setup Project after the LLM gives you the EditorScript. This creates all folders at once.

Phase 2: The Genetics System

This is the core of the game. Implement in order — each file depends on the previous.

2.1 — Gene Definition (ScriptableObject)

Ask your LLM:

```
"Create a Unity ScriptableObject called GeneDefinition.cs in
Scripts/Genetics/.
It should have: string geneId, string displayName, GeneType type (enum:
Continuous,
Discrete), float minValue, float maxValue, int discreteStates (for discrete
genes),
float dominanceWeight (0-1, used in discrete inheritance), bool
isMutationSensitive,
float mutationSigma. Include a tooltip attribute on each field."
```

2.2 — The Fish Genome

Ask your LLM:

```
"Create FishGenome.cs in Scripts/Genetics/. This class (not MonoBehaviour)
represents
a fish's full genetic data. It should contain: a Dictionary<string, float[]>
called
alleles (key = geneId, value = float[2] for the two alleles). Include
methods:
float GetPhenotype(string geneId, GeneDefinition def) — returns expressed
value
(average for continuous, dominance-resolved for discrete),
static FishGenome Breed(FishGenome parentA, FishGenome parentB, GeneLibrary
lib) —
picks one allele per parent per gene, then calls MutationSystem.Mutate(),
string Serialize() and static FishGenome Deserialize(string json) for cloud
saves,
float GeneticDistanceTo(FishGenome other) — Euclidean distance across all
genes."
```

2.3 — Gene Library

Ask your LLM:

```
"Create GeneLibrary.cs in Scripts/Genetics/ as a ScriptableObject. It holds a
List<GeneDefinition> of ALL genes in the game. Include a method
GeneDefinition GetGene(string geneId) with a cached Dictionary for O(1)
lookup.
Also include FishGenome GenerateRandomGenome() for starter fish creation."
```

□ **AUTOMATED** — Ask your LLM: "Write a Unity 6 EditorScript at Editor/GeneDefinitionsGenerator.cs with menu item Tools/Aquarium/Generate All Gene Definitions. It should: (1) create a GeneDefinition ScriptableObject asset for every gene in the Phase 2.4 table inside Assets/ScriptableObjects/Genes/GeneDefinitions/, (2) create a GeneLibrary asset at Assets/ScriptableObjects/Genes/DefaultGeneLibrary.asset and populate its genes list with all created definitions. Create missing folders automatically via AssetDatabase.CreateFolder. Use Unity 6 APIs." Then run: Tools > Aquarium > Generate All Gene Definitions.

2.4 — Full Gene List to Create as ScriptableObject Assets

Create one GeneDefinition asset per row. These are all the genes in the game:

Gene ID	Type	Range / States	Mutation Sigma	Notes
body_shape	Discrete	8 states	—	0=oval 1=elongated 2=deep 3=flat 4=round 5=torpedo 6=ribbon 7=diamond
body_size	Continuous	0.4 – 2.0	0.08	Affects speed cap and hitbox
tail_type	Discrete	6 states	—	0=forked 1=fan 2=lyre 3=delta 4=ribbon 5=spade
fin_count	Discrete	3 states	—	1, 2, or 3 dorsal fins
fin_length	Continuous	0.5 – 3.0	0.10	Multiplied by tail base size
fin_shape	Discrete	5 states	—	0=rounded 1=pointed 2=rayed 3=filamentous 4=tattered
eye_size	Continuous	0.3 – 1.8	0.06	Small=predatory, large=cute
eye_color_h	Continuous	0 – 360	15.0	Hue of eye iris
lip_type	Discrete	4 states	—	0=normal 1=pucker 2=overbite 3=tube
base_hue	Continuous	0 – 360	20.0	Primary body color hue
base_saturation	Continuous	0.0 – 1.0	0.07	
base_value	Continuous	0.3 – 1.0	0.06	Brightness
pattern_type	Discrete	9 states	—	0=solid 1=striped 2=spotted 3=marbled 4=gradient 5=iridescent 6=outlined 7=reticulated 8=banded
pattern_hue	Continuous	0 – 360	18.0	Secondary color; constrained to harmony
pattern_scale	Continuous	0.2 – 2.0	0.09	Size/density of pattern elements
pattern_contrast	Continuous	0.0 – 1.0	0.07	Blend vs. sharp edges

Gene ID	Type	Range / States	Mutation Sigma	Notes
iridescence	Continuous	0.0 – 1.0	0.05	Metallic shimmer. >0.7 = visually striking
bioluminescence	Continuous	0.0 – 1.0	0.04	Glow radius. Rare — see mutation section
transparency	Continuous	0.0 – 0.6	0.04	>0.4 = ghost fish effect
temperament	Continuous	-1.0 – 1.0	0.08	-1=shy/hides, +1=aggressive/chases
swim_style	Discrete	5 states	—	0=straight 1=sinusoidal 2=darting 3=hovering 4=spiral
depth_preference	Continuous	0.0 – 1.0	0.06	0=surface, 1=bottom
school_tendency	Continuous	0.0 – 1.0	0.07	Flocking behavior strength
lifespan	Continuous	0.5 – 2.0	0.06	Multiplied against 21-day base
fertility	Continuous	0.0 – 1.0	0.06	Breeding success rate modifier
hardiness	Continuous	0.0 – 1.0	0.05	Resistance to tank neglect
growth_rate	Continuous	0.5 – 2.0	0.07	Time to reach adult size
rarity_class	Discrete	5 states	—	0=Common...4=Legendary. Partially emergent

2.5 — Mutation System

This is where rare new traits emerge. Ask your LLM:

```
"Create MutationSystem.cs in Scripts/Genetics/. Static class with method:
static void Mutate(FishGenome genome, GeneLibrary lib, float
globalMutationRate = 0.03f).
For each gene: if Random.value < globalMutationRate, apply mutation.
Continuous: add Gaussian noise (Box-Muller transform) scaled by
gene.mutationSigma.
Clamp result to [minValue, maxValue].
Discrete: jump to a random adjacent state (±1) with wraparound.
Also implement: static bool TriggerHypermuation(FishGenome genome,
GeneLibrary lib)
- called with 0.2% probability per birth. Applies 5x mutation sigma to ALL
genes.
Returns true if triggered (caller should set a HypermutantFlag on the
fish).
Also implement: static bool CheckPhenotypeLock(FishGenome genome,
GeneLibrary lib)
- returns true if: iridescence>0.8 AND pattern_type==5 AND
base_saturation<0.2.
When true, set bioluminescence alleles to min(current+0.3, 1.0). This is
the
```

```
hidden Rainbow Ghost unlock condition."
```

⚠ The PhenotypeLock check creates hidden emergent traits players discover and share. Add more locks in future updates — they're powerful engagement drivers.

2.6 — Breeding Manager

Ask your LLM:

```
"Create BreedingManager.cs in Scripts/Genetics/ as a MonoBehaviour singleton.
It manages: List<BreedingSlot> activeSlots (max 4 slots, start with 2
unlocked).
BreedingSlot has: FishData parentA, FishData parentB, DateTime startTime,
float durationHours, bool isComplete, FishGenome pendingOffspring.
Methods:
    bool TryStartBreeding(FishData a, FishData b) - checks slot availability,
    calculates genetic distance, applies compatibility score (see notes),
    calls FishGenome.Breed(), stores result with timer.
    void Update() - checks DateTime.UtcNow against slot timers, fires
    OnEggReady(BreedingSlot slot) event when complete.
    float GetCompatibilityScore(FishGenome a, FishGenome b) -
    base: 1.0 - (geneticDistance * 0.5). If body_shape differs by 3+,
multiply
    by 0.4. If geneticDistance < 0.15, apply inbreeding: multiply fertility
    gene effect by 0.5 in offspring.
    Breeding duration = Mathf.Lerp(2f, 8f, 1f - compatibilityScore) hours."
```

Phase 3: Fish Data & Lifecycle

3.1 — FishData Class

```
"Create FishData.cs in Scripts/Fish/ (not MonoBehaviour – plain serializable class).  
Fields: string fishId (GUID), string ownerId, FishGenome genome,  
string lineageTag (auto-generated name), DateTime birthTime, DateTime deathTime,  
bool isAlive, bool isPreserved (jar display mode), bool isHypermutant,  
string parentAId, string parentBId, int generationNumber,  
List<string> offspringIds, float currentAge (0-1 normalized to lifespan),  
FishLifeStage stage (enum: Egg, Fry, Juvenile, Adult, Elder).  
Include: float GetBaseLispanDays() => 21f * genome.GetPhenotype('lifespan', lib).  
string GenerateLineageTag() – combine body_shape name + dominant color name + random 3-letter suffix."
```

3.2 — Fish Lifecycle Manager

```
"Create FishLifecycleManager.cs in Scripts/Fish/ as a singleton MonoBehaviour.  
It should: track all FishData in the player tank via List<FishData> tankFish.  
Every real-time minute, call UpdateAges() – increment currentAge based on elapsed real time vs. baseLisfpanDays.  
Stage transitions (currentAge thresholds): Egg=0, Fry=0.05, Juvenile=0.20, Adult=0.35, Elder=0.80.  
When currentAge >= 1.0: call KillFish(fishId) – sets isAlive=false, records deathTime, fires OnFishDied event. Does NOT delete data (lineage tree).  
Fry care window: between Egg->Fry transition, player has 24 real hours to 'feed' the fry (call FeedFry(fishId)). If window expires unfed: apply a permanent -0.2 penalty to lifespan and hardiness gene phenotype display (store as a FishData.neglectPenalty float, applied at render/stat time)."
```

Phase 4: Visual Rendering System

The visual system renders each fish procedurally from its genome. It has three parts: the pattern shader, the body morph system, and the layer compositor.

4.1 — Fish Pattern Shader

Ask your LLM:

```
"Create a Unity URP ShaderGraph shader called FishPattern.shadergraph (or write a HLSL shader if ShaderGraph is unavailable) with these properties:
_BaseColor (Color), _PatternColor (Color), _PatternType (Float, 0-8),
_PatternScale (Float), _PatternContrast (Float), _Iridescence (Float),
_Bioluminescence (Float), _Transparency (Float), _Time (Float, auto).
Pattern functions to implement in HLSL:
    stripes(uv, scale): sin(uv.y * scale * 6.28) * 0.5 + 0.5
    spots(uv, scale): distance from nearest point in a jittered grid
    marble(uv, scale): sin((uv.x + uv.y + fbm(uv*scale)) * 6.28) * 0.5 + 0.5
    gradient(uv): uv.x
    reticulated(uv, scale): 1 - spots (inverted)
    iridescent: add sin(dot(viewDir, normal) * 3.14 + _Time) * _Iridescence * 0.4
    to final color HSV value.
    bioluminescence: add an additive glow pass; output emission =
        _BaseColor * _Bioluminescence * (sin(_Time*2.0)*0.2+0.8) * 2.0
    transparency: set alpha = 1.0 - _Transparency in the output."
```

❏ AUTOMATED — Ask your LLM: "Write a Unity 6 EditorScript at Editor/FishShaderGenerator.cs with menu item Tools/Aquarium/Generate Fish Pattern Shader. Use System.IO.File.WriteAllText to write a complete hand-coded HLSL shader (not ShaderGraph) to Assets/Shaders/FishPattern.shader implementing the URP Unlit shader with these Properties: _BaseColor, _PatternColor, _PatternType (Float 0-8), _PatternScale, _PatternContrast, _Iridescence, _Bioluminescence, _Transparency. Implement all 9 pattern functions (stripes, spots, marble, gradient, iridescent, outlined, reticulated, banded, solid) in the fragment shader selected by _PatternType. Include additive emission for bioluminescence and alpha for transparency. Call AssetDatabase.Refresh() after writing." Then run: Tools > Aquarium > Generate Fish Pattern Shader.

4.2 — Fish Body Mesh Morphing

```
"Create FishBodyMorpher.cs in Scripts/Rendering/. It should:
Reference 8 Mesh assets (bodyMeshes[0..7]) assigned in Inspector.
Method: Mesh GetMorphedMesh(float bodyShapeGene, float bodySizeGene) -
    int stateA = Mathf.FloorToInt(bodyShapeGene)
```



```

int stateB = Mathf.CeilToInt(bodyShapeGene)
float t = bodyShapeGene - stateA
Lerp vertex positions between meshes[stateA] and meshes[stateB]
Scale all vertices by bodySizeGene
Return new Mesh with lerped vertices, same triangles as base mesh."

```

❑ **YOU DO THIS: Create 8 base fish body meshes in a 2D art tool (Illustrator, Aseprite, or use a mesh editor in Unity). They should share the same vertex count and topology so lerping works. Export as .obj or use Unity's built-in Sprite Shape. This is the most art-intensive step in the project. Consider hiring a 2D artist for this one task.**

4.3 — Fish Renderer Component

```

"Create FishRenderer.cs in Scripts/Rendering/ as a MonoBehaviour.
It manages the full render stack for one fish. Inspector refs:
  SpriteRenderer[] layerRenderers (8 layers, back to front)
  Material patternMaterial (instance of FishPattern shader)
  FishBodyMorpher bodyMorpher, GeneLibrary geneLib
Method: void ApplyGenome(FishData fish) –
  1. Get morphed body mesh via bodyMorpher
  2. Set patternMaterial properties from genome phenotypes:
      mat.SetColor('_BaseColor', HSVToRGB(base_hue, base_saturation,
base_value))
      mat.SetFloat('_PatternType', pattern_type phenotype)
      mat.SetFloat('_PatternScale', pattern_scale phenotype)
      [... all other shader properties ...]
  3. COLOR HARMONY ENFORCEMENT: before setting _PatternColor,
      float hueDiff = Mathf.Abs(pattern_hue - base_hue)
      if (hueDiff > 30 && hueDiff < 60) pattern_hue = base_hue + 65 (push to
safe zone)
  4. PROPORTION GUARD: if body_shape == 6 (ribbon), cap fin_length at 1.5
  5. Apply fin sprite (swap sprite based on tail_type integer)
  6. Apply eye sprite (scale by eye_size, tint by eye_color_h)
  7. Apply glow layer (set emission intensity from bioluminescence gene)"

```

4.4 — Fish Animation Controller

```

"Create FishAnimator.cs in Scripts/Rendering/. It drives swimming animation
using swim_style and temperament genes. No Animator component – use code.
Update() method: move fish position and rotation each frame.
  swim_style 0 (straight): move in facing direction, gently wobble rotation
±5°
  swim_style 1 (sinusoidal): position += facing * speed; add
sin(Time*freq)*amplitude to perpendicular axis

```

```
swim_style 2 (darting): random bursts – idle for 0.5-2s, then fast dash  
0.3s  
swim_style 3 (hovering): nearly still, gentle bob on Y axis  
swim_style 4 (spiral): circular path with slow drift  
temperament affects: negative = fish avoids tank edges and other fish;  
positive = fish moves toward other fish (use steering behavior).  
school_tendency: if >0.5, apply simple flocking (alignment + cohesion +  
separation)  
toward nearest 3 fish with school_tendency > 0.5."
```

Phase 5: Game Loop & Daily Retention

5.1 — Tank Manager

```
"Create TankManager.cs in Scripts/GameLoop/ as a singleton MonoBehaviour.
It is the central game state. Manages: List<FishData> allFish,
int tankCapacity (start: 10, max: 50), float tankHealth (0-1),
DateTime lastFeedTime, int playerCurrency, int premiumCurrency.
tankHealth degrades: -0.1 per real hour if no fish fed; feeding restores
+0.3.
If tankHealth < 0.3: all fish get -0.05 daily to hardiness phenotype
display.
Events to fire: OnFishAdded, OnFishDied, OnEggHatched, OnTankHealthChanged.
Serialize entire state to JSON for cloud save via
string SerializeState() / void DeserializeState(string json)."
```

5.2 — Notification Scheduler

```
"Create NotificationScheduler.cs in Scripts/GameLoop/. At key events,
schedule local Unity push notifications (use Unity Mobile Notifications
package):
    When egg starts: schedule notification at hatchTime - 15 minutes: 'Your
    egg is almost ready!'
    When fry hatches: schedule notification at hatchTime + 20 hours: 'Your fry
    needs feeding soon!'
    When fish enters Elder stage: 'One of your fish is aging - breed it soon!'
    When tankHealth < 0.3: 'Your tank needs attention!'
Use Unity.Notifications.Android and Apple namespaces with #if directives."
```

❑ **AUTOMATED — Ask your LLM: "Write a Unity 6 EditorScript at Editor/PackageInstaller.cs with menu item Tools/Aquarium/Install Required Packages. Use `UnityEditor.PackageManager.Client.Add()` to sequentially install these package IDs: `com.unity.mobile.notifications`, `com.unity.services.authentication`, `com.unity.services.cloudsave`, `com.unity.services leaderboards`, `com.unity.services.economy`, `com.unity.services.remoteconfig`. Show a progress dialog with `EditorUtility.DisplayProgressBar` and log success/failure for each." Then run: Tools > Aquarium > Install Required Packages and wait for completion.**

5.3 — Daily Login & Streak System

```
"Create DailyLoginManager.cs in Scripts/GameLoop/. Tracks:
DateTime lastLoginDate, int currentStreak, int longestStreak.
On app open: if lastLoginDate.Date < DateTime.UtcNow.Date, it is a new day.
Increment streak. Award daily reward based on streak tier:
    Day 1-6: 50 currency. Day 7: 200 currency + random starter fish."
```

```
Day 14: rare wild fish injection (high mutation gene values).
Day 30: legendary fish (bioluminescence > 0.7, random rare traits).
Wild fish injection: generate a FishGenome with GenerateRandomGenome(),
then call MutationSystem.Mutate() 5 times on it before giving to player.
Show a modal UI popup with the reward using UnityEvent
OnDailyRewardReady(RewardData)."
```

5.4 — Stud System (Player Economy)

```
"Create StudRegistry.cs in Scripts/GameLoop/. Players can mark fish as public
studs.
StudEntry: fishId, ownerId, ownerName, FishGenome genome snapshot, ',
    int breedingFee (set by owner), int timesUsed, DateTime listedAt.
Methods: ListAsStud(FishData fish, int fee), RemoveFromStud(string fishId),
    BreedWithStud(StudEntry stud, FishData myFish) - deducts fee from player,
    awards fee to stud owner (via cloud transaction), calls
FishGenome.Breed(),
    returns the offspring genome without removing the stud fish from owner's
tank.
This entire class should communicate with Unity Gaming Services Leaderboards
and Economy."
```

Phase 6: Unity Gaming Services Integration

⚠ This phase requires a Unity account and UGS project setup at dashboard.unity3d.com. Create a project, enable Authentication, Cloud Save, Leaderboards, and Economy.

- ❑ **YOU DO THIS:** Go to dashboard.unity3d.com > Create Project > Enable services: Authentication, Cloud Save, Leaderboards, Economy. Copy your Project ID. In Unity: Edit > Project Settings > Services > link your project.
- ❑ **AUTOMATED** — The `PackageInstaller` script from Phase 5.2 already installs all UGS packages. If you skipped it, run Tools > Aquarium > Install Required Packages now.

6.1 — Auth & Save Service

```
"Create CloudSaveService.cs in Scripts/Services/. Wraps Unity Gaming Services.
Init: await UnityServices.InitializeAsync(); await
AuthenticationService.Instance.SignInAnonymouslyAsync().
SaveTank(): serialize TankManager state to JSON, call
CloudSaveService.Instance.Data.Player.SaveAsync(new
Dictionary<string,object>{{'tankState', json}}).
LoadTank(): load and deserialize. Call both on app start and every 5
minutes.
Handle exceptions: if save fails, queue locally and retry on next app open."
```

6.2 — Leaderboards

```
"Create LeaderboardService.cs in Scripts/Services/. Manage three leaderboards
(configure these IDs in UGS dashboard first: rarest-fish, lineage-depth,
phenotype-diversity).
SubmitScore(string leaderboardId, double score): call
LeaderboardsService.Instance.AddPlayerScoreAsync(leaderboardId, score).
GetTopScores(string leaderboardId, int count): returns LeaderboardEntry
list.
Scoring functions:
RarestFishScore: sum of (rarity_class phenotype * 10) for all live fish
LineageDepthScore: max generationNumber across all fish
PhenotypeDiversityScore: count of distinct discretized genome signatures"
```

6.3 — Visit Other Players' Tanks

```
"Create TankVisitService.cs in Scripts/Services/. When visiting a friend's
tank:
Load their tank state from Cloud Save (requires their playerId).
Instantiate their fish as read-only FishRenderer objects in a VisitScene."
```

Player can: view fish details, request to breed with a stud fish (triggers StudRegistry flow),
leave a 'like' (stored in their cloud save, max 1 per visitor per day)."

Phase 7: UI Screens

Ask the LLM to generate each screen separately. Each is a Canvas prefab.

Screen	Prompt Summary	Key Components
Main Tank View	Scrollable tank with all fish swimming. Tap fish to inspect.	FishRenderer per fish, ScrollRect, Tap detection, Health bar
Fish Inspector	Shows fish stats, genome visualization, lineage tree up 3 generations.	Gene bar chart, parent portraits, rarity badge
Breeding UI	Drag two fish to breeding slots, shows compatibility score, starts timer.	DragDrop handlers, CompatibilityMeter, Timer countdown
Egg Hatchery	Shows active eggs with time remaining. Tap to hatch when ready.	BreedingSlot cards, countdown timers, hatch animation trigger
Lineage Tree	Visual tree of a fish's ancestry. Nodes are small fish portraits.	Recursive tree layout, portrait thumbnails, generation labels
Phenotype Registry	Scrollable list of all player-discovered phenotype combos.	ScrollView, PhenotypeCard, discovery date, first-discoverer badge
Leaderboard	Three tabs for the three leaderboard types. Shows rank, name, score.	LeaderboardService calls, TabGroup, animated rank reveal
Stud Market	Browse public studs, filter by body type and rarity, buy breeding.	Grid layout, Filter dropdowns, StudCard prefab, purchase flow
Daily Reward	Modal popup for daily login reward with animated fish gift.	Modal canvas, reward animation, claim button
Settings	Mute audio, notification toggles, link account.	Standard toggles, CloudSaveService links

UI Build Prompt Template

Use this template for each screen:

```
"Create a Unity UI Canvas prefab for [SCREEN NAME]. Style: dark aquarium theme, deep blue-black background (#0A1628), accent color teal (#00D4AA), text color white (#F0F8FF), font size body=16 header=22. Use Unity UI (not UIToolkit). All buttons use Button component with [describe layout and components from table above]."
```

Phase 8: Gene Pool Anti-Collapse Systems

Without these, all fish in a player's tank become nearly identical after 10+ generations.

8.1 — Diversity Monitor

```
"Create GenepoolMonitor.cs in Scripts/Genetics/. Runs weekly (real time).
  Calculates: for each continuous gene, mean and variance across all living
  fish.
  If variance < 0.05 for any gene (collapsed): set a DriftAlert flag on that
  gene.
  If 3+ genes are flagged: spawn a WildFish event -
    generate a random genome, mutate it 3x, add to tank as a free 'wild
  visitor'
    with a notification: 'A wild fish has found your tank!'
  Store last-run timestamp in PlayerPrefs to survive app restarts."
```

8.2 — Outbreeding Bonus

```
"In BreedingManager.TryStartBreeding(), after calculating genetic distance:
  if (distance > 0.5): mutationRate *= 1.5f for this offspring (more
  variation),
    award 10% fertility bonus (increase success chance by 0.1).
  if (distance < 0.15): apply inbreeding coefficient -
    offspring lifespan gene alleles both -= 0.1,
    set fish.isInbred = true, show warning icon in UI."
```

8.3 — Hidden Phenotype Lock Conditions

These are the 'secret discoveries' that drive community sharing. Add to MutationSystem.CheckPhenotypeLocks():

Name	Unlock Condition	Effect When Triggered
Rainbow Ghost	iridescence>0.8 AND pattern_type==5 AND base_saturation<0.2	bioluminescence boosted to min(current+0.3, 1.0)
Veil Tail	tail_type==4 (ribbon, recessive) AND fin_shape==3 (filamentous, recessive)	fin_length multiplied by 1.5 at render time
Abyssal Form	depth_preference>0.9 AND base_value<0.35 AND transparency>0.3	swim_style forced to 3 (hovering); glow flicker added
Solar Fish	bioluminescence>0.6 AND base_hue between 30-60 (yellow/orange) AND base_value>0.9	iridescence boosted +0.2; emission color shifts to warm gold

Name	Unlock Condition	Effect When Triggered
Predator Mark	temperament>0.9 AND eye_size<0.5 AND body_shape==5 (torpedo)	pattern_contrast boosted to 1.0; movement speed cap increased 20%

Add new phenotype locks in future updates without any other system changes — they're fully self-contained.

Phase 9: Rarity & Discovery Systems

9.1 — Rarity Scoring

```
"Create RarityScorer.cs in Scripts/Genetics/. Static method:
int CalculateRarityClass(FishData fish, GeneLibrary lib):
    score = 0
    if bioluminescence > 0.7: score += 2
    if transparency > 0.4: score += 1
    if iridescence > 0.8: score += 1
    if tail_type == 4 (ribbon, recessive): score += 2
    if fish.isHypermutant: score += 1
    if any phenotype lock is active: score += 2
    if generationNumber > 10: score += 1
    return Mathf.Clamp(score/2, 0, 4) // maps to 0=Common..4=Legendary
Call this after every hatch and store result in fish.rarityClass.
Never let players manually set rarity - always computed."
```

9.2 — Phenotype Registry (Discovery Log)

```
"Create PhenotypeRegistry.cs in Scripts/Genetics/. On each fish hatch,
compute a PhenotypeSignature: discretize all continuous genes to 4 buckets,
concatenate with discrete gene values into a compact hash string.
Check cloud (via CloudSaveService) if this signature has been discovered
globally.
If not: mark as new discovery, record playerName + timestamp,
award DiscoveryBonus currency, show 'FIRST DISCOVERY' modal.
If yes: show 'Already discovered by [name] on [date]'.
Store global registry in UGS Cloud Save under a shared key (not per-
player)."
```

Phase 10: Build & Launch Checklist

- ☐ **AUTOMATED** — Ask your LLM: "Write a Unity 6 EditorScript at Editor/BuildConfigurator.cs with two menu items: Tools/Aquarium/Configure Android Build and Tools/Aquarium/Configure iOS Build. Each should call EditorUserBuildSettings.SwitchActiveBuildTarget(), set PlayerSettings.applicationIdentifier, PlayerSettings.productName, PlayerSettings.bundleVersion, enable il2cpp scripting backend, and set target architectures (ARM64 for both). Log a checklist of remaining manual steps (Apple certificate, google-services.json) after running." Then run the appropriate menu item for your target platform.
- ☐ **YOU DO THIS:** Configure UGS in dashboard: set up the 3 leaderboards (rarest-fish, lineage-depth, phenotype-diversity), Economy currencies (coins, gems), and Remote Config keys for seasonal event multipliers.
- ☐ **AUTOMATED** — Ask your LLM: "Add a static bool BreedingManager.debugFastBreeding and a menu item Tools/Aquarium/Toggle Fast Breeding that flips it. When true, treat all duration hours as seconds. Display an obvious red label in the Game view when active using OnGUI so it is never shipped accidentally."
- ☐ **YOU DO THIS:** Set up push notification certificates: iOS requires an APNs certificate in Apple Developer Portal. Android uses FCM — add google-services.json to your project.

Area	What to Check
Performance	Profile on a mid-range Android (not just top-end). Target 60fps. Fish shader should run 20+ fish at once.
Genetics	Run 50 generations in an automated test — check gene pool doesn't collapse. Verify hypermutation fires ~1 in 500 births.
Save/Load	Kill the app mid-game. Reopen — all fish, timers, and currency should be exactly as left.
Notifications	Test on real device — simulator notifications are unreliable. Verify egg hatch and fry care notifications fire correctly.
Edge Cases	Two fish with identical genomes breeding (inbreeding path). Tank at max capacity. All breeding slots full.
Localization	Dates and times use device locale for display (not UTC strings).

Recommended Build Order

Build and test in this order — each phase is playable before the next:

1. GeneDefinition + GeneLibrary + FishGenome (data layer, no visuals yet)
2. MutationSystem + BreedingManager (test in a console/editor test script)
3. FishRenderer + FishPattern shader (get one fish looking good visually)
4. FishAnimator + TankManager (fish swimming in a tank)
5. FishLifecycleManager + aging/death (fish that live and die)

6. Breeding UI + egg hatchery (full breeding loop playable)
7. GenepoolMonitor + RarityScorer + PhenotypeRegistry
8. UGS integration (CloudSave, Auth, Leaderboards)
9. StudRegistry + tank visits (multiplayer features)
10. Notifications + DailyLogin (retention layer)
11. Polish: UI screens, sound, particle effects, store page

Good luck — the genetics system is genuinely fun once the first fish start looking different.